MODULE 01 · DOMAIN 1 - DESIGN APPLICATIONS

The shape of a GenAI application

The parts every later module assumes, what each one is for, and where it lives on Databricks.

SECTION 1 of 6 EXAM WEIGHT 14% MODULE 01 of 40

admin@sthithapragna.com

A model on its own knows what it was trained on, up to a date, and nothing about your company. **Everything else is what you build around it.**

What a model alone cannot do, and what fixes it

THE MODEL ALONE CANNOT	SO YOU ADD
See your private documents	A retriever over an index of them
Know what changed last week	A retriever that stays in sync with the source
Look up a live number or take an action	Tools it can call
Remember the user from last Tuesday	Memory stored outside the model
Be trusted not to say something harmful	Guardrails on input and output
Prove it is getting better	Evaluation and monitoring

ON THE EXAM

Diagnose the gap before choosing the fix. A retrieval problem is not solved by a bigger model.

A generative AI application is a newsroom

IN THE NEWSROOM	IN THE APPLICATION
The reporter who writes the story	The LLM that writes the answer
The researcher who pulls the files	The retriever and its index
The editor's brief	The prompt template
The phone and the database terminal	Tools
The reporter's notebook	Memory
The legal desk	Guardrails
The fact-checker	Evaluation

ON THE EXAM

A brilliant reporter with a bad researcher still files a wrong story. Most RAG failures are retrieval failures.

What happens when a user asks a question

01
Ask
The user's question arrives.

02
Embed
The question becomes a vector.

03
Retrieve
The index returns the closest chunks.

04
Augment
Chunks are placed into the prompt.

05
Generate
The LLM writes a grounded answer.

ON THE EXAM

Retrieval augmented generation changes what the model can see, not what the model is. No weights change.

Rag has an offline half and an online half

OFFLINE, BEFORE ANYONE ASKS

Build the index

- Choose and collect source documents
- Extract text from each format
- Split into chunks
- Embed chunks and store in an index

ONLINE, EVERY REQUEST

Answer the question

- Embed the user's question
- Retrieve the closest chunks
- Build the prompt with context
- Generate, then check the output

THE TIEBREAKER

If the fix happens before any user arrives, it is offline: section 2. If it happens per request, it is online: sections 3 and 4.

A **compound AI system** gets its results from several interacting components, not from one model call.

Words the exam uses without defining

TERM	MEANING
Chunk	A passage of a document, small enough to embed and to fit in a prompt
Embedding	A vector that represents meaning. Similar meaning, nearby vectors
Index	A searchable store of embeddings plus their text and metadata
Chain	Steps wired together by the developer in a fixed order
Agent	An LLM that decides which tool to call next, and when to stop
Tool	A function the model can ask to run: a query, a lookup, an action
Judge	An LLM used to score another LLM's output

ON THE EXAM

Chain and agent are the pair to get exactly right. The difference is who decides the next step.

A continuum, from fixed to autonomous

01

Single LLM call

A prompt in, an answer out. No retrieval.

02

Deterministic chain

Developer fixes the steps and their order.

03

Single agent

The LLM chooses tools and loops until done.

04

Multi-agent

A supervisor delegates to specialist agents.

ON THE EXAM

Databricks' own advice: start simple, and add autonomy only when the task needs model-driven decisions.

Chain or agent?

CHOOSE A CHAIN WHEN

The steps are known in advance

- Every request follows the same path
- Predictability and audit matter most
- Latency and cost must stay low
- Failures should be easy to trace

CHOOSE AN AGENT WHEN

The next step depends on results

- The path varies by request
- A lookup result changes what to do next
- Several tools exist and the model must pick
- You accept more latency for flexibility

THE TIEBREAKER

If you could draw the flowchart before seeing the request, build a chain. If the flowchart depends on what comes back, build an agent.

Retrieval or fine-tuning?

REACH FOR RETRIEVAL WHEN

The problem is knowledge

- The model lacks your facts
- The facts change often
- Answers must cite a source
- Different users may see different documents

CONSIDER FINE-TUNING WHEN

The problem is behaviour

- Prompting cannot fix a style or format
- You have many good labelled examples
- The knowledge itself is stable
- You accept retraining when things change

THE TIEBREAKER

Knowledge problem: retrieve. Behaviour problem: prompt first, fine-tune last. Fine-tuning to add fresh facts is almost always a distractor.

Where each part lives

PART	DATABRICKS FEATURE
The model	Model Serving and Foundation Model APIs
Document retrieval	AI Search (formerly Vector Search)
Questions over tables	Genie Agents (formerly Genie Spaces)
Orchestration code	Custom Agents (formerly Agent Framework) with MLflow
Tools	Unity Catalog functions and MCP servers
Memory	Lakebase, managed Postgres
Traffic control and logging	Unity AI Gateway

ON THE EXAM

Quality is measured with MLflow evaluation and tracing. Underneath everything sits Unity Catalog, governing data, indexes, functions, models and prompts.

Harbourline Mutual's policy assistant

01
Question
Am I covered for storm damage to a fence?

02
Retrieve
Policy wording chunks from AI Search.

03
Look up
The customer's policy tier from a table.

04
Answer
Grounded in wording, cited, checked.

05
Log
Request, response and trace recorded.

ON THE EXAM

One request touched a retriever, a tool, a model, a guardrail and a log. That is a compound AI system.

> An insurer launched a policy assistant built on retrieval over its policy documents. The index was built once, at launch, from a folder of PDFs.
>
> Three months later, customers report that the assistant quotes an excess amount that was changed last month. The new PDFs are already in the folder.

What should the engineer change?

A Move to a larger LLM with a more recent training cut-off

B Fine-tune the LLM on the updated policy documents each month

C Keep the index in sync with the source documents as they change

D Add an instruction to the prompt to always use the latest figures

◆ PAUSE AND DECIDE

What should the engineer change?

A Larger LLM, newer cut-off

Public training data never contained this insurer's excess. Wrong gap.

B Fine-tune monthly

Freezes facts into weights, costs a retrain per change, and still lags.

C Keep the index in sync

The model only sees what retrieval returns. Stale index, stale answer.

D Prompt: use the latest

The model cannot use figures it was never shown.

THE SIGNAL

The new PDFs are already in the folder. That sentence says the source is fine and the index is stale.

> Every Monday, a claims team wants a summary email. It must pull last week's closed claims from a table, summarise the common causes, and format them into a fixed template.
>
> The steps never change. The compliance team must be able to audit exactly what ran, and the cost should stay as low as possible.

Which design fits best?

A. A deterministic chain: query, then summarise, then format

B. A tool-calling agent that decides which steps to run

C. A multi-agent system with a supervisor and three specialist agents

D. A model fine-tuned on past summary emails

◆ PAUSE AND DECIDE

Which design fits best?

A Deterministic chain

Fixed steps, fixed order, cheapest and fully auditable.

B Tool-calling agent

Would work, but pays for decisions nobody needs and is harder to audit.

C Multi-agent supervisor

Most complexity and latency for the simplest flow on the page.

D Fine-tuned model

Cannot query last week's claims. It has no access to the table.

THE SIGNAL

The steps never change. When you could draw the flowchart in advance, the answer is a chain.

A retailer wants a chatbot that answers customer questions using its product manuals. There are about four thousand manuals, updated when products change.

The team plans a standard retrieval augmented design using a hosted foundation model.

Which component is NOT required for this application?

A. An embedding model for chunks and for questions

B. An index over the chunked manuals

C. A prompt template that inserts retrieved chunks

D. A language model pre-trained from scratch on the manuals

◆ PAUSE AND DECIDE

Which component is NOT required for this application?

A Embedding model

Required. Without it there is nothing to compare the question against.

B Index over chunks

Required. This is what retrieval searches.

C Prompt template with context

Required. This is the augment step itself.

D Model pre-trained from scratch

Not required. A hosted foundation model already does generation.

THE SIGNAL

Read NOT twice. Then eliminate each option by asking which rag step it serves. The one that serves no step is the answer.

> An insurer's claims assistant must look up a claim's status. If the claim has been pending for more than thirty days, it must open an escalation ticket.
>
> Otherwise it should explain the status to the customer using the relevant policy wording. Each conversation may take a different path.

Which design meets the requirement?

A. A retrieval chain over the policy documents and nothing else

B. An agent with tools for claim lookup, tickets and policy retrieval

C. A model fine-tuned on several years of past claim conversations

D. A nightly batch job that scores each open claim with a large LLM

◆ PAUSE AND DECIDE

Which design meets the requirement?

A Retrieval chain only

Can explain wording, but cannot read a claim or open a ticket.

B Agent with three tools

The lookup result decides the next action. That is an agent's job.

C Fine-tuned on old chats

Learns how past chats sounded, not this claim's live status.

D Nightly batch scoring

Not interactive, and a day stale. The customer is waiting now.

THE SIGNAL

If it has been pending for more than thirty days... The next step depends on what a tool returns. That is the definition of an agent.

What to carry forward

- Diagnose the **gap** first: knowledge, freshness, action, memory, safety or evidence.
- Rag has an **offline** half, section 2, and an **online** half, sections 3 and 4.
- Chain when the flowchart is known in advance. **Agent when it depends on results.**
- Start simple. Databricks says so, and so do the exam's right answers.
- Fine-tuning to add fresh facts is almost always the distractor.

> ON THE EXAM
>
> Next: module 02 turns a business requirement into model tasks and a clear input and output contract.